

Corporate Access & Intelligence System

Complete Technical Architecture & Interview Preparation Guide

Elevator Pitch for Technical Interviewers:

'This project is an Enterprise Workforce & Multi-Million Row CSV Analytics Platform built with React 18, Vite, Chart.js, and HTML5 Web Workers. It enables users to upload massive CSV datasets (up to 10+ million rows) and analyze them in real-time right inside the browser. By offloading parsing, schema detection, statistical profiling, and filtering to a dedicated background Web Worker thread, the application maintains a silky-smooth 60 FPS UI without freezing the DOM. It also features zero-trust multi-method authentication (SSO, OAuth 2.0, Email, Mobile OTP), custom chart building, statistical profiling, and multi-group comparative analytics.'

1. How It Works in the Background (Backend & Worker Architecture)

Since this is a high-performance client-side application, the core processing engine runs inside a dedicated **HTML5 Web Worker thread** (src/workers/csvWorker.js). This keeps the main browser UI thread completely unblocked.

Key Architectural Highlights:

- **Chunk-Based Streaming Parsing (PapaParse):** Files are streamed in 4MB memory chunks. As parsing progresses, intermediate row counts are sent back to the main thread via worker messages, displaying real-time loading UI without browser lag.
- **Sampled Schema & Type Auto-Detection:** The worker samples the first 5,000 rows. Columns with >80% numeric values are classified as *numeric*, >80% date strings as *datetime*, and remaining columns as *categorical*.
- **Single-Pass Fast Statistical Profiling:** Computes Mean, Median, Standard Deviation, Min, Max, missing counts, frequency maps, and 5-number box-plot quartiles (Min, Q1, Median, Q3, Max) in a single pass over the dataset.
- **Data Health Score Engine:** Evaluates overall dataset integrity index: $Health\ Score = \max(0, 100 - (missingCells / totalCells) * 100)$.
- **Off-Thread Filtering & Pagination Memory Management:** The master array stays inside worker memory. When users apply search keywords or range filters, filtering executes off-thread. Only lightweight aggregated metrics (for Chart.js) and active 10-100 row table page slices are posted back to the UI thread.

2. Frontend Architecture & Component Breakdown

The frontend is crafted with **React 18** and **Vite**, styled with custom Vanilla CSS design tokens supporting Dark & Light themes (data-theme), CSS Grid layouts, and glassmorphism styling.

Component File	Core Responsibility & UI Features
App.jsx	Central state coordinator. Manages theme state, authentication checks, Web Worker instantiation (new Worker), filter state, and active tab routing.
LoginPage.jsx & LoginModal.jsx	Enterprise authentication portal. Supports Email/Password, Mobile SMS OTP verification, demo account auto-fill, and guest exploration mode.

OAuthPromptModal.jsx	Simulates Enterprise OAuth 2.0 & SAML SSO handshakes with Google Workspace, Microsoft 365 / Azure AD, GitHub Enterprise, and Okta SAML SSO.
Header.jsx	Brand navbar showing dataset indicator badges, sample dataset loader, CSV export triggers, theme toggle button, and logged-in user profile menu.
KPICards.jsx	Top metric cards displaying Total Records, Filtered Overview (with instant LOW / MED / HIGH / ALL level preset buttons), Avg Highlight Metric, and Health Score.
SidebarFilters.jsx	Interactive sidebar featuring global keyword search, categorical multi-select checkboxes with counts, and numeric min-max sliders with quick level presets.
Dashboard.jsx	Executive dashboard rendering 6 Chart.js charts: Bar, Doughnut, Box Plot Whisker, Histogram, Time-Series Area, and Radar Spider profile.
CustomChartBuilder.jsx	Custom Visual Studio allowing dynamic axis selection (X & Y), aggregation functions (mean, sum, count, min, max), and chart types (bar, line, doughnut).
DataTable.jsx	High-performance paginated table with column sorting (asc/desc), row page size dropdown (10-100), status badges, and filtered CSV exporter.
StatsOverview.jsx	Statistical profiling matrix presenting per-column data types, missing counts/percentages, mean, median, std dev, min, max, and unique categories.
ComparisonView.jsx	Dynamic CSV group comparison studio with side-by-side group cards, category leaderboards, comparative bar charts, and heatmaps.

3. Data Models & Data Flow

Model Name	Storage Location	Data Structure & Description
Master Dataset Model	csvWorker.js memory	Full array of parsed CSV objects: [{ Department: 'Engineering', Salary: 95000, Status: 'Active' }, ...]
Schema Model	Worker & React State	Column type dictionary: { Department: 'categorical', Salary: 'numeric', JoinDate: 'datetime' }
Stats Profiling Model	Worker & React State	Numerical & Categorical metrics: { count, missingCount, mean, median, stdDev, min, max, uniqueCount, topCategory, frequencies }
Quartile Box-Plot Model	Worker Dashboard Metrics	5-number summary per category: { min, q1, median, q3, max } for whisker box plots.
Filter State Model	React State (App.jsx)	Filter payload: { search: "", categorical: { Dept: ['Sales'] }, numeric: { Salary: [50000, 120000] } }
User Session Model	localStorage & auth.js	Auth profile object: { id, name, email, role, avatar, loginType, provider, loginTime }

4. Tools, Libraries & Tech Stack

- **Frontend Core:** React 18, Vite 5, PapaParse 5
- **Multithreading:** HTML5 Web Workers API (Dedicated Worker thread)
- **Data Visualization:** Chart.js 4 & react-chartjs-2 (Bar, Line, Doughnut, Radar)
- **Iconography:** Lucide React
- **Styling & Theme:** Custom Vanilla CSS3 Design Tokens, Dark/Light Mode (data-theme), Glassmorphism, CSS Grid

- **Storage & Persistence:** HTML5 Web Storage API (localStorage)

5. Technical Interview Q&A; Cheat Sheet

Q1: Why did you choose Web Workers for this project?

Standard JavaScript runs on a single main UI thread. If you parse and aggregate a 500,000+ row CSV on the main thread, the browser UI will freeze, causing drop-down lag, unresponsive buttons, and browser warning dialogs. By using an HTML5 Web Worker, we move parsing, schema inference, statistical calculations, and filtering off the main thread. The UI stays 100% responsive at 60 FPS while the worker handles heavy computation in the background.

Q2: How do you handle memory optimization when handling large datasets?

We use three strategies: First, chunk-based parsing via PapaParse streams 4MB chunks rather than loading entire raw file strings at once. Second, the full master dataset stays in the worker's isolated thread memory. Third, when filtering or paginating, the worker only sends the current 10-100 row page slice and small pre-aggregated metric objects to the UI thread via `postMessage`, drastically reducing DOM memory overhead.

Q3: How does the authentication flow work without a traditional backend server?

We implemented a modular zero-trust client-side authentication utility (`auth.js`). It supports multi-factor Email/Password and Mobile SMS OTP flows, as well as simulated OAuth 2.0 / SAML SSO handshakes for Google Workspace, Microsoft Azure AD, GitHub Enterprise, and Okta. User sessions are persisted securely in `localStorage` with role-based access control.

Q4: How does the Dynamic Comparison Studio work with arbitrary CSV files?

The comparison engine dynamically inspects the schema of whatever CSV file is loaded. It isolates low-cardinality categorical columns as group dimensions (e.g., Department, Work Mode, Region) and numeric columns as metrics (e.g., Salary, Rating). It computes multi-group statistics, determines category champions for the leaderboard, and generates side-by-side bar charts and heatmaps on the fly.